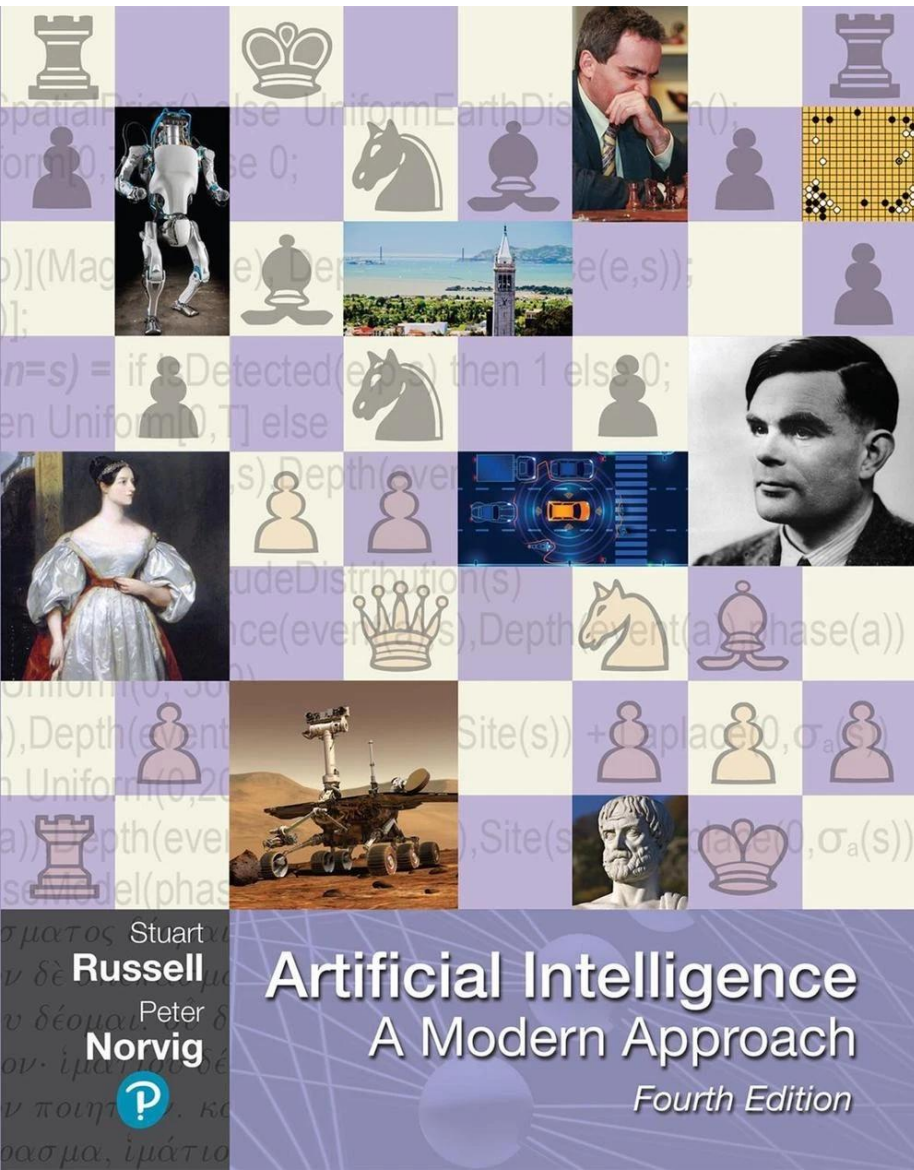


Chapter 8

Reinforcement Learning Part II



Reinforcement Learning – Part II



- Model-Free Learning: Active Learning
- Q-Learning
- Approximate Q-Learning
- Policy Search

Model-Free Learning: Active Learning



Introduction to Active RL

- **Active RL:** The agent actively chooses its actions while interacting with the environment.
- Unlike Passive RL, the policy is not fixed in advance.
- The agent learns by trying different actions, observing the resulting rewards and state transitions, and improving its behavior over time.
- Typical methods in Active RL include TD control, Q-learning, and policy-based methods.

Objectives of Active RL

- Learn an optimal policy that maximizes the expected cumulative reward over time.
- Balance exploration and exploitation:
 - Exploration helps discover new information about the environment.
 - Exploitation uses current knowledge to gain higher rewards.
 - To avoid sticking to a sub-optimal policy.
- Improve decision making efficiently, even when the environment is initially unknown or uncertain.

Epsilon-Greedy Strategy

- An action selection strategy that balances exploration and exploitation.
 - With probability ϵ , the agent selects a random action to explore.
 - With probability $1 - \epsilon$, the agent selects the action with the highest estimated value.
- Allows the agent to mostly follow what it has learned, but still retains a small probability of trying new actions.
- Prevents the agent from getting stuck in a local solution by only utilizing current knowledge.

Q-Learning



Introduction to Q-Learning

- **Definition:** Q-learning is a **model-free** active reinforcement learning method.
- **Core Approach:** Instead of learning state utilities $U(s)$, the agent learns an **action-utility function** $Q(s, a)$.
- **Main Advantage:** The agent **does not need a transition model** $P(s'|s, a)$ to make decisions or learn.

The Action-Utility Function

- **Definition of $Q(s,a)$:** The expected total discounted reward of taking action a in state s and acting optimally thereafter.
- **Q (Quality)** represents the quality of the pair (State, Action).
- **Optimal Decision:** Once Q is learned, the agent simply chooses the action with the highest Q value: $\pi^*(s) = \operatorname{argmax}_a Q(s, a)$.
- **Constraint:** Q -values are related to utilities by $U(s) = \max_a Q(s, a)$.
- For example, at state s , $Q(s, \text{Left}) = 2$ and $Q(s, \text{Right}) = 5$. The agent will choose Right because this action has a higher expected quality.

The Q-Learning Update Rule

- **TD Update:** Q-learning uses a **Temporal-Difference (TD)** approach.

- **The Formula:** Whenever action a is taken in s leading to s' :

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- **Components:**

- TD error: $R + \gamma \max_{a'} Q(s', a') - Q(s, a)$

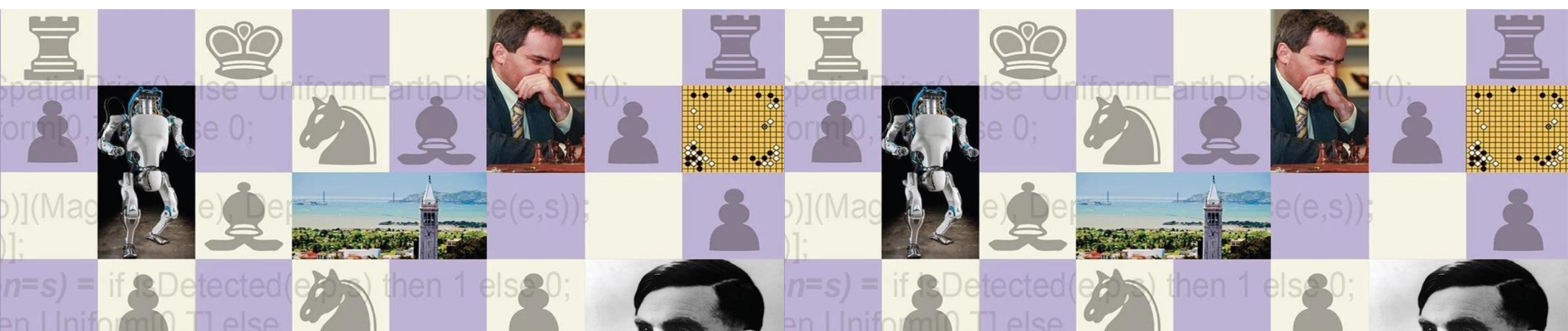
- α is the learning rate
- γ is the discount factor
- R is the reward

Example: $Q(s, a) = 0.5$, $R = 1$, $\gamma = 0.9$, $\max Q(s', a') = 0.8$ and $\alpha = 0.1$. Then the target TD = $1 + 0.9 \times 0.8 = 1.72$, error TD = 1.22 , new $Q = 0.5 + 0.1 \times 1.22 = 0.622$.

Why Q-Learning is Important

- Q-Learning can learn optimal behavior without a model.
- It is simple, flexible, and widely used as a foundation for advanced methods such as Deep Q-Networks.

Approximate Q-Learning



Motivation

- Q-Learning with a table does not scale to large or continuous state spaces.
- Approximate Q-Learning replaces the Q-table with **a function approximator**.
- The goal is to generalize knowledge across similar states and actions.
- For example, a robot has real-world coordinates (x, y) , velocity, and rotation angle. Because these values are continuous, we use features such as distance to the target instead of creating a table for every coordinate.

What is Approximate Q-Learning

- Approximate Q-Learning estimates $Q(s, a)$ using a parameterized function:

$$Q(s, a) = \theta \cdot f(s, a)$$

- Instead of storing values, it learns a set of parameters (θ).
- Learning is based on experience and gradient-based updates.

Linear Function Approximation

- **Feature-based Representation:** Approximate the utility or Q-value using a weighted linear combination of features $f_1, \dots, f_n$.

- **Equation:**

$$Q_{\theta}(s, a) = \theta_1 f_1(s, a) + \theta_2 f_2(s, a) + \dots + \theta_n f_n(s, a)$$

- **Benefit:** Learning just a few parameters (θ) allows the agent to generalize to states it has never seen.

Approximate Q-Learning Update Rule

- **Mechanism:** Adjust parameters θ to reduce the temporal difference error.

- **Update Rule:**

$$\theta_i \leftarrow \theta_i + \alpha [R + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s, a)] \frac{\partial Q_{\theta}(s, a)}{\partial \theta_i}$$

- Temporal Difference error: $R + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s, a)$
- α : Learning Rate - Controls the size of the parameter adjustment.

From Approximate Q-Learning to Deep Q-Learning

- **Linear approximation** is limited in expressiveness.
- **Neural networks** can be used as **powerful function approximators**.
- This leads to **Deep Q-Learning** and **Deep Reinforcement Learning**.

Case Study: TD-Gammon

- **Architecture:** Uses a fully connected neural network with one hidden layer of 80 nodes.
- **Training:** Trained over 300,000 games using temporal-difference learning.
- **Result:** Reached a standard of play comparable to the top three human players in the world.

Policy Search



Introduction to Policy Search

- **Policy** = the way an agent chooses an action in a given state
- **Policy Search** is a RL method that **learn the policy directly**.
- **The policy maps states to actions** without explicitly learning a utility function.
- **The objective is to optimize policy parameters** to maximize expected reward.

Why Policy Search

- Value-based methods can struggle with large or continuous action spaces.
- Policy Search handles continuous actions naturally.
- It can learn stochastic policies directly.

Policy Representation

- A policy can be parameterized by a set of parameters θ .
- Policies can be deterministic or stochastic:
 - A deterministic policy: always choose the same action in a given state.
 - A stochastic policy: choose actions according to a probability distribution, allowing different actions to be selected in the same state.
- Common representations include tables, linear models, and neural networks.

Policy Gradient Methods

- Policy Gradient methods optimize policies using **gradient ascent**.
- **The gradient** is computed from sampled trajectories.
- **REINFORCE** is a classic policy gradient algorithm.

Exploration in Policy Search

- Exploration is achieved naturally through stochastic policies.
- Action probabilities encourage trying different actions.
- No explicit epsilon-greedy strategy is required.

Advantages and Limitations

■ Advantages:

- Simplicity and suitability for continuous control
- Policies can be optimized directly for performance

■ Limitations:

- High variance: each run can produce very different results, leading to unstable updates.
- Sample inefficiency: a large number of episodes is required to learn a good policy.

Summary (1/2)

- **RL:** Learning from **rewards or punishments** to maximize the expected sum of rewards over time.
- **Model-based RL:** The agent attempts to learn or is equipped with a **transition model** $P(s'|s, a)$ and a **reward function** $R(s, a, s')$.
- **Model-free RL :** The agent **neither knows nor learns a transition model** $P(s'|s, a)$ of the environment.
- **Passive RL:** Agents have a **fixed policy**; the goal is to learn the utility function $U^\pi(s)$ of each state.
- **Active RL:** Agents learn and actively choose their actions. Policies are not fixed but are gradually improved through interaction with the environment.

Summary (2/2)

- **Exploration vs. Exploitation:** A rational agent must balance **exploitation** (choosing the best known action) and **exploration** (trying unknown states to gain information).
- **Q-learning** learns the utility of actions ($Q(s, a)$) **without needing a transition model** of the environment.
- **Deep RL:** Uses **deep neural networks** as function approximators to handle vaster state spaces, like those in Go or video games.
- **Reward Shaping:** Providing **intermediate pseudorewards** to help the agent learn when true rewards are sparse.

